

Where does everything live?

1

A 5-minute refresher built from your actual questions. Start here, then follow the message.

“Is a broker my laptop?”

A **broker** is a **Kafka server process**. Ours runs inside Docker on your laptop. A **cluster** is the group of brokers working together; this lab has one.

“Three partitions = three servers?”

No. Our one broker holds **all three partitions**. Partitions are separate ordered logs stored on disk, not three computers or three disk partitions.

YOUR LAPTOP

Python process

Your code calls **produce()**



Client buffer + background sender
Both live in this process.

network

Docker: broker 1

vessel.positions / partition 0

vessel.positions / partition 1

vessel.positions / partition 2

acknowledgement returns

“Thousands of vessels, only three partitions?”

The producer hashes the MMSI key to choose a partition. **Many vessels share each partition**. Three keys do not guarantee three different partitions. Same key stays in the same partition while partition count and routing stay unchanged.

KEY = ROUTING

The key decides **where** a record goes.

Example: A and C can share partition 0; B can use partition 1; partition 2 can be empty.

ORDER = PER PARTITION

Kafka preserves append order **within a partition**. There is no single global order across all partitions. Late source events can still arrive out of time order.

Queue. Send. Wait.

The background sender belongs to the Python client library. It is NOT the Docker broker.

produce() = enqueue

Puts the event into the client's memory buffer. Background threads handle delivery. **Returning from produce() does not mean Kafka has received it.**

flush() = wait here

Waits for outstanding deliveries and serves delivery callbacks. It gives messages time to finish **before your code moves on or exits**. It is not required to start sending.

```
produce(message_1)    # queue quickly
produce(message_2)    # queue quickly
produce(message_3)    # queue quickly
remaining = producer.flush(timeout=15)
```

Conceptual example: the three produce calls above stand for your `producer.produce(...)` calls.

“Does flush wait 15 seconds for EACH message?”

No. **15 seconds is one maximum wait for all outstanding messages**. If they finish in 0.2 seconds, flush returns then. If records remain at the deadline, it returns their count. Putting flush after each send makes your code wait between sends.

“Why flush if delivery happens without it?”

A short script can exit while messages are still buffered. Exiting kills its sender threads too. **flush creates a waiting point before shutdown**. A long-running app usually calls `poll()` regularly for callbacks, then `flush()` when shutting down.

Two clocks, two jobs

flush(timeout=15)

How long this call blocks: up to 15 seconds.
Timeout stops waiting, not delivery.

message.timeout.ms=10000

Client delivery limit: 10 seconds before reporting failure.

Read the callback!

remaining == 0 means nothing is pending. Some deliveries may have failed.

The callback reports success or failure. **poll()** and **flush()** run callbacks; the sender threads perform network delivery.

Same event, new offset?

Sequence is yours. Offset is Kafka's. Key is for routing. Event ID is for application identity.

"Is sequence the same as offset?"

sequence is a field our generator creates (0-4 per vessel). **offset** is the record's location within one Kafka partition. Offsets are not unique across the whole topic.

YOUR ACTUAL RUN

MMSI key	Partition	Our sequence	Kafka offsets
999000001	1	0, 1, 2, 3, 4	1, 2, 3, 4, 5
999000002	2	0, 1, 2, 3, 4	0, 1, 2, 3, 4
999000003	0	0, 1, 2, 3, 4	0, 1, 2, 3, 4

Vessel 1 starts at offset 1 because your first experiment already occupied offset 0.

"Does Kafka deduplicate?"

Not by your key or event_id. Sending the same JSON twice normally appends **two records at different offsets**.

Rerun our script: sequence and event_id repeat; offsets continue.

Two kinds of duplicates

Internal retry: producer idempotence can protect against retry duplicates when enabled.

New send / script rerun: a new record. Later, our database will enforce a unique event_id.

"What if Python exits before delivery?"

1. Still in the local buffer: unsent messages disappear with process memory. Restarting the script does not restore them.

2. Sent, but no acknowledgement yet: Kafka may have stored the message. The producer cannot be certain; a new send could duplicate it.

3. Successful acknowledgement: Python exiting does not remove the stored record. Broker durability is separate; our one-node lab has no replica redundancy.

Back to the lab

Session checkpoint: one broker, three partitions, generator + producer working. Consumer is next.

“What does isoformat() do?”

Python datetime becomes a standard **string**:
 2026-10-04T00:00:01+00:00
 T separates date/time; +00:00 means UTC. Time is unchanged.

JSON has no native datetime type. Use **timestamp.isoformat()** before `json.dumps(event)`.

“What does my output mean?”

partition=1 offset=0
 Kafka stored the record at that location. **It does not mean a consumer read it.**

In topic description: Leader: 1, Replicas: 1, Isr: 1 are **broker IDs**. ReplicationFactor: 1 is the copy count.

```
# From ~/my_projects/vessel-operations-copilot
uv python pin 3.12.8
uv add confluent-kafka
docker compose up -d
uv run python producer.py

# Inspect the topic
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe --topic vessel.positions
```

Setup problems we actually solved

Port 9092 occupied: an existing kind Kubernetes container owned it.

UnknownHostException: fixed container hostname + local host mapping.

Missing process.roles: used complete explicit single-node broker/controller settings.

Keep the final working compose.yaml. A container being “Started” is not readiness.

ONE MONTH LATER: cover the answers

1. Three vessels and three partitions: one vessel each?
2. Can messages send without `flush()`? Why keep it?
3. Does `flush` returning zero prove delivery succeeded?
4. Rerun the generator: which values repeat, which increase?
5. Does the Docker broker die when Python exits?

Answers: 1 No. 2 Yes; wait before exit. 3 No; check callbacks. 4 IDs/sequences repeat; offsets increase. 5 No; only Python’s client and its threads stop.

Keep in mind: reading does not delete Kafka records; retention governs removal. Our lab has no explicit persistent volume: do not rely on container recreation preserving data.